

MixCraft

IA Generative pour la Recommandation et Creation de Cocktails

Questions & Reponses - Preparation Soutenance

Projet : M1 Data Engineering & IA - EFREI Paris

Auteurs : Adam Beloucif & Emilien Morice

Periode : Decembre 2025 / Fevrier 2026

Sections : 6 themes - 28 questions

Metriques cles du projet

P@5 SBERT + RAG	0.79	+93% vs <i>TF-IDF</i> baseline
BLEU-4 generation	0.42	coherent avec la litterature
Guardrail F1	0.92	94% rejet OOD - seuil 0.40
Latence retrieval	45ms	cache hit <10ms
Corpus final	612 cocktails	4 sources Kaggle + CocktailDB

1. Architecture et choix techniques

Q1. Pourquoi SBERT plutôt que TF-IDF ou Word2Vec ?

TF-IDF est un modèle sac-de-mots sans contexte : 'citron' et 'lime' sont deux tokens indépendants. Word2Vec capture la sémantique mais produit un embedding moyen des mots, insensible à l'ordre. SBERT (all-MiniLM-L6-v2) produit des embeddings de phrases entières en 384 dimensions via un encodeur Transformer, L2-normalise pour la similarité cosinus. Résultat concret : $P@5 = 0.71$ avec SBERT vs 0.41 avec TF-IDF, soit +73%.

Q2. Pourquoi all-MiniLM-L6-v2 et pas un modèle plus grand comme all-mpnet-base-v2 ?

Choix de frugalité computationnelle : all-MiniLM-L6-v2 est 5x plus rapide (45ms vs 220ms par requête) pour une perte de performance négligeable sur notre domaine clos. Le modèle tient en RAM (80 MB), ce qui permet un déploiement local sans GPU, compatible avec un environnement académique. Sur des benchmarks MTEB spécialisés domaine étroit, la différence entre MiniLM et MPNet est <3%.

Q3. Comment fonctionne l'index FAISS IndexFlatIP ?

FAISS (Facebook AI Similarity Search) est une bibliothèque de recherche de voisins approches dans des espaces vectoriels. IndexFlatIP calcule le produit scalaire entre la requête et tous les vecteurs du corpus (bruteforce exact). Pour des vecteurs L2-normalisés, le produit scalaire est équivalent à la similarité cosinus. À 612 cocktails, la recherche exacte est instantanée (<1ms). Pour un corpus de 100k+, on passerait à IndexIVFFlat (clustering + recherche approchée).

Q4. Pourquoi GPT-2 small et pas un LLM moderne comme llama ou mistral ?

GPT-2 small (117M paramètres) est fine-tunable localement sur CPU en 30 minutes avec un dataset de 612 exemples. Les LLM modernes (7B+) nécessitent un GPU avec 16GB+ VRAM et ne se fine-tunent pas sur du matériel standard. Dans un cadre académique avec hardware Intel i7 sans GPU, GPT-2 est le bon compromis. En production, on utiliserait LoRA sur Mistral-7B ou l'API d'un fournisseur externe.

Q5. Qu'est-ce que RAG et pourquoi l'utiliser ici ?

RAG (Retrieval-Augmented Generation, Lewis et al. 2020) combine deux étapes : (1) Retrieval : on récupère les K exemples les plus pertinents du corpus par similarité sémantique. (2) Generation : on injecte ces exemples dans le prompt comme contexte pour guider la génération. L'avantage : réduit les hallucinations en ancrant la génération dans des données réelles. Sans RAG, GPT-2 fine-tune peut générer des ingrédients improbables. Avec RAG + top-3 références, les recettes générées restent cohérentes : $P@5$ passe de 0.71 à 0.79.

Q6. Comment le système construit le texte d'embedding ?

La concaténation est : nom + ' ' + 'Category: ' + catégorie + ' ' + 'Ingredients: ' + ingrédients + ' ' + instructions. Ce format place le nom en premier (fort signal sémantique), suivi des ingrédients (profil de saveurs) et des instructions (contexte de préparation). Le choix d'inclure les instructions capture des informations implicites : un 'shake' vs un 'stir' change le style du cocktail.

2. Donnees et preprocessing

Q7. Comment avez-vous nettoye et fusionne les 4 datasets Kaggle ?

Etapes de preprocessing : (1) Harmonisation des schemas : chaque source a des colonnes differentes (strIngredient1..15 pour CocktailDB, ingredient_list pour d'autres). On normalise vers le schema [name, category, glass, ingredients, instructions]. (2) Fusion des ingredients en une seule chaine 'mesure ingredient, ...' pour homogeneiser. (3) Deduplication par cle name.lower().strip() apres tokenisation. Resultat : 612 cocktails uniques sur 1470 lignes brutes (42% de duplicats entre sources).

Q8. Pourquoi 612 cocktails ? Ce n'est pas beaucoup pour un systeme de recommandation.

C'est une limite reconnue dans nos resultats (section 7.2 du rapport). 612 est la taille du corpus apres deduplication inter-sources. Un systeme commercial aurait besoin de 5000+ entrees. Cependant, pour valider l'architecture RAG + SBERT, 612 exemples suffisent : la precision topologique de l'espace d'embedding se manifeste des 100 exemples. En perspective, TheCocktailDB API publique fournit 600 recettes supplementaires (deja integrees).

Q9. Comment avez-vous calcule le score silhouette K=8 pour le clustering ?

Le clustering KMeans est applique sur les embeddings SBERT (612 x 384 matrix). Le score silhouette mesure la coherence intra-cluster vs la separation inter-cluster : $s(i) = (b(i) - a(i)) / \max(a(i), b(i))$ avec a = distance intra, b = distance au cluster voisin. Notre score de 0.31 sur K=8 indique des clusters moyennement distincts (0.5+ serait excellent). Cela reflète la nature continue de l'espace cocktails : les frontieres entre categories sont floues (un 'Tropical Shot' appartient aux clusters 0 et 2 simultanément).

Q10. Quelle est la distribution des langues dans le corpus ? Comment gerez-vous le francais ?

Le corpus est quasi-exclusivement en anglais (CocktailDB, Kaggle). SBERT all-MiniLM-L6-v2 est entraine principalement sur l'anglais. Pour les requetes en francais, on observe une degradation d'environ 15% de P@5. La solution identifiee en perspective : utiliser LaBSE (Language-Agnostic BERT Sentence Embeddings) ou XLM-RoBERTa qui supportent 100+ langues avec des performances quasi-equivalentes a MiniLM.

3. Evaluation et metriques

Q11. Comment avez-vous construit le jeu de test pour evaluer P@5 ?

Nous avons cree un benchmark de 30 requetes de test annotees manuellement : 15 requetes avec ground truth explicite (ex. 'cocktail tropical rhum' -> [Pina Colada, Mai Tai, Blue Lagoon]) et 15 requetes ambiguës pour tester les cas limites. $P@5 = (\text{nombre de resultats pertinents dans les 5 premiers}) / 5$, moyenne sur 30 requetes. Les annotations ont ete faites par les deux auteurs independamment puis reconciliees (taux d'accord inter-annotateurs $\kappa=0.78$).

Q12. Que mesurent BLEU-4 et ROUGE-L pour la generation ?

BLEU-4 (Bilingual Evaluation Understudy) mesure la precision des 4-grammes : proportion de 4-grammes de la generation presentes dans les references. ROUGE-L mesure le rappel du sous-sequence commune la plus longue. Notre BLEU-4 = 0.42 et ROUGE-L = 0.58 sont coherents avec la litterature (RecipeNLP : BLEU-4 typiquement 0.35-0.55). Limite importante : ces metriques ne capturent pas la coherence gustative. Une recette peut avoir un BLEU-4 faible mais etre objectivement bonne.

Q13. Comment avez-vous calibre le seuil du guardrail a 0.40 ?

Nous avons constitue un jeu de 30 requetes : 15 in-domain (sur les cocktails) et 15 hors-domain (cuisine, medecine, informatique, etc.). Le seuil est le score de similarite max entre la requete et le corpus. On balaie les seuils de 0.20 a 0.60 et on mesure le F1 du rejet binaire. A 0.40 : $F1=0.92$ (Precision=0.95, Recall=0.90). En dessous de 0.35 : trop de faux rejets (requetes legitimes rejetees). Au-dessus de 0.50 : trop de faux positifs (requetes OOD acceptees).

Q14. Quelle est la limite principale de votre evaluation ?

L'evaluation quantitative (BLEU, ROUGE, P@5) ne mesure pas la qualite subjective. Un cocktail 'Dark Tempest' (bourbon + vermouth + bitters) peut scorer faible en BLEU si les references sont differentes, mais etre gustativement coherent. Une evaluation ideale inclurait : (1) un test utilisateur aveugle (blind tasting), (2) une evaluation par des experts (sommeliers, barmen), (3) une evaluation de la faisabilite (les ingredients existent-ils ?). Ces evaluations sont hors de portee d'un projet academique de 5 semaines.

4. Critique et limites

Q15. Votre systeme hallucine-t-il ? Comment le detectez-vous ?

GPT-2 sans RAG hallucine : il peut generer des ingredients inexistantes ('Dragon Syrup', '90ml of Moonfire'). Avec RAG, les hallucinations sont reduites car la generation est conditionnee par des recettes reelles. Cependant, notre systeme ne valide pas que les ingredients generes existent dans le corpus. Solution partielle : notre ingredient_index.parquet (772 ingredients connus) pourrait servir de garde-fou post-generation pour filtrer les ingredients invalides. C'est une piste d'amelioration.

Q16. Le cache MD5 est-il une optimisation reelle ou juste un patch ?

C'est une optimisation legitime et largement utilisee en production (ex. Redis cache pour LLM). La cle MD5 est calculee sur (model_name + texte_requete), ce qui garantit l'unicite. L'impact est mesurable : latence cache hit <10ms vs ~2.1s sans cache (x210). La limite est la memoire disque : sans TTL ni eviction, le cache croit indefiniment. En production, on utiliserait Redis avec TTL=24h et taille max.

Q17. Pourquoi ne pas avoir utilise une API comme GPT-4 ou Gemini pour la generation ?

Trois raisons : (1) Budget : pas de cle API disponible pour le projet academique. (2) Pedagogique : l'objectif est de comprendre le fine-tuning, pas de wrapper une API. (3) Scientifique : l'utilisation d'une API externe rend l'experience non-reproductible et dependante d'un service tiers. Notre pipeline est 100% local et reproductible. En contexte professionnel, une API serait le bon choix pour la generation finale.

Q18. Comment justifiez-vous le choix de Streamlit pour l'interface ?

Streamlit est adapte aux prototypes data science : quelques lignes Python suffisent pour une interface interactive. Le projet avait 5 semaines, dont 2 dediees au pipeline IA. L'alternative React/Next.js aurait pris 2 semaines supplementaires pour la meme fonctionnalite. Streamlit a aussi l'avantage d'etre adopte dans le monde data (Data Apps Streamlit Community). La limite : pas de rendu mobile, personnalisation CSS limitee (contournee via st.markdown + injection CSS).

Q19. 62 tests pytest, c'est suffisant ? Quelle est la couverture ?

62 tests couvrent les cas critiques des composants metier : data_loader (10 tests), embeddings (12 tests), recommender (15 tests), rag_pipeline (15 tests), generator (10 tests). La couverture fonctionnelle n'a pas ete mesuree formellement avec pytest-cov, mais les happy paths et les cas limites (corpus vide, requete vide, guardrail trigger) sont tous couverts. En production, on viserait 80%+ de couverture avec pytest-cov.

5. Perspectives et industrialisation

Q20. Comment scaler ce systeme a 100 000 cocktails ?

Trois adaptations : (1) Remplacer IndexFlatIP par FAISS IndexIVFFlat ou HNSW pour une recherche approuchee en $O(\log n)$ au lieu de $O(n)$. (2) Stocker les embeddings dans une base vectorielle (pgvector, Weaviate, Qdrant) avec persistance et mises a jour incrementales. (3) Servir les embeddings via une API (FastAPI + uvicorn) et un worker pool pour paralleliser les requetes. Le generateur passerait a un LLM quantize (llama.cpp, GGUF).

Q21. Quels problemes ethiques pose un systeme de recommandation de cocktails alcoolises ?

Questions ethiques : (1) Le systeme ne verifie pas l'age de l'utilisateur. En production, une validation legale (18+/21+) est obligatoire selon les juridictions. (2) Les recommandations de cocktails forts pourraient encourager la consommation excessive. Un garde-rail 'responsabilite' pourrait limiter les recommandations $> 40\%$ ABV. (3) Les donnees de consommation utilisateur (historique de requetes) sont sensibles. RGPD : retention minimale, chiffrement, droit a l'oubli. (4) Biais de corpus : si le corpus surrepresente certaines cultures (cocktails americains), le systeme sous-recommande les boissons traditionnelles d'autres regions.

Q22. Comment amelioreriez-vous les embeddings pour le domaine cocktails ?

Fine-tuning de SBERT par domain adaptation : (1) Constituer un jeu de paires (cocktail A, cocktail B, score de similarite) annote par des experts barmen. (2) Fine-tuner all-MiniLM-L6-v2 avec une loss contrastive (triplet loss) pour que les cocktails de meme famille soient proches et les dissimilaires eloignes. (3) Valider sur le benchmark P@5 : on estime un gain de 10-15% par rapport a SBERT generique (comme observe dans la litterature domain-adapted SBERT).

Q23. Quel serait le cas d'usage industriel realiste pour MixCraft ?

Trois scenarios : (1) Application bar connecte : tablette integree a la caisse, recommande des recettes selon les stocks disponibles et les preferences du client. (2) Plateforme B2C (type Vivino pour les cocktails) : recommandation personnalisee basee sur l'historique de l'utilisateur avec filtrage collaboratif hybride. (3) Outil de formation : aide les jeunes barmen a apprendre les associations d'ingredients par exemple similaire. L'interface Streamlit actuelle serait remplacee par une application mobile React Native avec backend FastAPI.

6. Questions de jury / pieges classiques

Q24. La Precision@5 de 0.79 a ete mesuree sur votre propre jeu de test. Est-ce biaise ?

Risque de surapprentissage sur le jeu de validation reconnu. Notre protocole d'evaluation a deux gardes-fous : (1) Le jeu de test (30 requetes) n'a pas ete utilise pendant le developpement du systeme, uniquement pour l'evaluation finale. (2) Les seuils (guardrail=0.40, K=5) ont ete calibres sur un sous-ensemble separe des donnees de validation, pas sur le jeu de test final. Un protocole plus rigoureux utiliserait une validation croisee k-fold avec des annotateurs externes independants.

Q25. Quelle est la difference entre votre RAG et un simple retrieval avec template ?

Distinction importante : un retrieval pur retournerait directement les K recettes existantes. Notre RAG utilise les recettes retrievees comme contexte pour generer une NOUVELLE recette, distincte des recettes de reference. La nouveaute se mesure : le nom et les instructions de la recette generee ne correspondent a aucun cocktail du corpus (verified par string matching). Le modele generatif (GPT-2 fine-tune) interpole entre les exemples fournis, produisant une recette originale ancrée dans le style du domaine.

Q26. Pourquoi votre BLEU-4 de 0.42 est-il meilleur que la litterature cite (0.35) ?

La comparaison directe avec la litterature est hasardeuse car les conditions varient : taille du corpus, longueur des recettes, methode d'annotation des references. Nos references sont les cocktails les plus proches dans le corpus (top-3 retrieval), ce qui favorise mecaniquement un BLEU eleve (les ingredients sont similaires). Une evaluation plus rigoureuse utiliserait des recettes humaines nouvelles comme references. C'est une limite methodologique que nous reconnaissons dans le rapport.

Q27. Qu'avez-vous appris de ce projet ? Que referiez-vous differemment ?

Apprentissages cle : (1) L'evaluation quantitative seule ne suffit pas pour les systemes creatifs. (2) La deduplication inter-datasets est sous-estimee (42% de duplicats non-triviaux). (3) Le guardrail doit etre calibre en premier, avant d'optimiser le retrieval. Ce que l'on referait : (1) Commencer par un benchmark d'evaluation solide avant de coder le systeme. (2) Utiliser MLflow pour tracker les experiences (seuils, modeles, hyperparametres). (3) Separer l'environnement de developpement (pyenv/venv) des le debut pour la reproductibilite.

Q28. Qui a fait quoi dans le binome ?

Division du travail : Adam Beloucif - pipeline de donnees (data_loader, download_datasets), moteur d'embeddings (EmbeddingEngine, cache MD5), interface Streamlit (3 onglets, CSS premium), evaluation (benchmark P@5, guardrail calibration), rapport final. Emilien Morice - pipeline RAG (RAGPipeline, prompt engineering), fine-tuning GPT-2 (CocktailGenerator, 3 epochs), evaluation generation (BLEU-4, ROUGE-L), tests pytest (62 tests). Points communs : EDA et clustering, revue de code mutuelle, preparation de la soutenance.